

DigitalBase 利用マニュアル

バックエンドは単一の **api**（API + frontend 同梱）に統合されており、Bare Metal は `~/.local/db` に1バイナリ、Docker は1イメージで配備します。vLLM と Ollama の切り替えは `.env` の `LLM_BACKEND` で行います。運用コマンドは Bare Metal が **db**、Docker は標準の **docker**（コンテナ名 **digitalbase-app**）です。

1. インストール方法

環境	コマンド	backend	インストール先
Linux	<code>install-linux.sh</code>	Ollama	<code>~/.local/db</code>
macOS	<code>install-macos.sh</code>	Ollama	<code>~/.local/db</code>
Windows	<code>install-windows.ps1</code>	Ollama	<code>%LOCALAPPDATA%\db</code>
Linux + GPU	<code>install-linux-vllm.sh</code>	vLLM	<code>~/.local/db</code>
Docker	<code>install-docker.sh</code>	vLLM / Ollama	<code>~/digitalbase</code> (<code>/app/data</code> に mount)
Kubernetes	Docker Hub の image を pull	vLLM / Ollama	任意 (Secret + PVC)

Bare Metal は単一バイナリ、Docker は単一 image です。edition は `.env` の `LLM_BACKEND` (`vllm` または `ollama`) で決まります。既定は Docker / GPU が `vllm`、Bare Metal デスクトップが `ollama` です。

Linux

```
curl -fsSL https://pub-a2cab4360f1748cab5ae1c0f12cddc0a.r2.dev/vite-scripts/install-linux.sh | bash
```

必要な環境: PostgreSQL (DB、pgvector 対応版・16 以降) / pgvector (ベクトル拡張・RAG 用) / Ollama (ローカル LLM ランタイム) / Tesseract OCR (画像・PDF の文字認識)

```
sudo apt install -y postgresql tesseract-ocr
sudo apt install -y postgresql-$(psql -V | grep -oE '[0-9]+' | head -1)-pgvector
# PG のバージョンに合わせる
curl -fsSL https://ollama.com/install.sh | sh
```

macOS

```
curl -fsSL https://pub-a2cab4360f1748cab5ae1c0f12cddc0a.r2.dev/vite-scripts/install-macos.sh | bash
```

必要な環境: PostgreSQL (DB、pgvector 対応版・16 以降) / pgvector (ベクトル拡張・RAG 用) / Ollama (ローカル LLM ランタイム) / Tesseract OCR (画像・PDF の文字認識)

```
brew install postgresql@17 pgvector ollama tesseract # postgresql@16 等、既存の対応版があればそれでよい
```

Windows

```
irm https://pub-a2cab4360f1748cab5ae1c0f12cddc0a.r2.dev/vite-scripts/install-windows.ps1 | iex
```

必要な環境: PostgreSQL (DB、pgvector 対応版) / pgvector (ベクトル拡張・RAG 用) / Ollama (ローカル LLM ランタイム) / Tesseract OCR (画像・PDF の文字認識)。いずれも installer が winget で導入します。管理者権限は不要で、通常ユーザーのまま実行できます。

pgvector (RAG 用) は、自前ビルド版 (VC++ Redistributable 不要) を自動配置します。ただし非管理者で実行した場合は RAG が無効化されます (警告のみで続行)。その場合は管理者で再実行するか、pgvector 同梱で管理者権限の要らない Docker 版を利用してください。

Linux (vLLM)

```
curl -fsSL https://pub-a2cab4360f1748cab5ae1c0f12cddc0a.r2.dev/vite-scripts/install-linux-vllm.sh | bash
```

必要な環境: PostgreSQL (DB、pgvector 対応版・16 以降) / pgvector (ベクトル拡張・RAG 用) / Tesseract OCR (画像・PDF の文字認識) / NVIDIA GPU + CUDA (vLLM は script が venv に自動導入します)

```
sudo apt install -y postgresql tesseract-ocr
sudo apt install -y postgresql-$(psql -V | grep -oE '[0-9]+' | head -1)-pgvector
# PG のバージョンに合わせる
```

GPU が必要で、初回起動時に HuggingFace から model を download します。インストール先や運用コマンドは Bare Metal Ollama 版と同じ (~/**.local/db/db** コマンド) で、統一バイナリです。**.env** の **LLM_BACKEND=vllm** によって vLLM として動作します。

Docker

Docker Hub で配布している単一イメージ **lmlight/digitalbase:latest** を使用します。vLLM 版・Ollama 版は共通のイメージで、エディションは **.env** の **LLM_BACKEND** によって切り替わります。PostgreSQL (pgvector) および LLM (vLLM / Ollama) はイメージに含まれないため、別途用意してください。

```
docker pull lmlight/digitalbase:latest
```

導入方法は次の2通りです。セットアップを自動化する「A. install-docker.sh」と、既存のインフラへ組み込む「B. 手動 docker run」のいずれかを選択します。

A. install-docker.sh (推奨)

イメージの取得、PostgreSQL (pgvector) コンテナとアプリケーションコンテナの起動、`.env` の生成までを自動で行います。

```
# vLLM 版 (既定)
curl -fsSL https://pub-a2cab4360f1748cab5ae1c0f12cddc0a.r2.dev/vite-
scripts/install-docker.sh | bash

# Ollama 版
curl -fsSL https://pub-a2cab4360f1748cab5ae1c0f12cddc0a.r2.dev/vite-
scripts/install-docker.sh | EDITION=ollama bash
```

- データの実体は任意のディレクトリ（既定は `~/digitalbase`、`DB_INSTALL_DIR` で変更可能）に保存し、コンテナの `/app/data` にマウントします。`.env` も同じディレクトリに配置されます。
- PostgreSQL (pgvector) を `digitalbase-postgres` コンテナとして、同一ネットワーク上に同時に起動します。
- 起動・停止・ログの確認には標準の `docker` コマンドを使用します (`docker logs -f digitalbase-app / docker stop digitalbase-app / docker start digitalbase-app`)。
- スクリプトが自動で行う処理は次のとおりです。① イメージの取得 (`docker pull`)、② `~/digitalbase/.env` の生成 (`JWT_SECRET / OAUTH_ENCRYPTION_KEY` を含む)、③ ネットワーク `digitalbase-net` の作成、④ PostgreSQL (pgvector) コンテナの起動と `digitalbase` ユーザー・データベースの作成 (拡張の有効化はアプリケーションが自動実行)、⑤ アプリケーションコンテナの起動 (`docker run`)、⑥ ライセンスファイル配置の案内。

B. 手動 docker run (既存インフラへの組み込み)

上記 A の処理のうち ①③④⑥ を手動で行う方法です。前提として、pgvector を導入済みの PostgreSQL (RAG 用) と、vLLM (ポート 8080 / 8081) または Ollama (ポート 11434) を用意してください。

拡張の有効化 (`CREATE EXTENSION vector`) およびスキーマ・テーブルの作成は、アプリケーションの起動時に自動で実行されます。pgvector 0.5 以降は trusted extension であるため、データベースの所有者である `digitalbase` ユーザーでも実行でき、スーパーユーザーは不要です (実行に失敗した場合は警告のみを出力し、RAG を無効化したうえで起動を継続します)。ただし、ユーザーとデータベースの作成だけは、事前にスーパーユーザーで行う必要があります (アプリケーションはユーザー・データベース自体を作成できません)。

PostgreSQL は、次のいずれかの方法で用意します。

```
# 方法1: PostgreSQL を別途用意しない場合は、pgvector 同梱イメージで起動する
#           (ユーザー・データベースも環境変数から自動的に作成される)
docker run -d --name digitalbase-postgres --restart unless-stopped \
  -e POSTGRES_USER=digitalbase -e POSTGRES_PASSWORD=digitalbase -e
POSTGRES_DB=digitalbase \
  -p 5432:5432 -v "$PWD/pgdata":/var/lib/postgresql/data \
  pgvector/pgvector:pg16

# 方法2: 既存の PostgreSQL を使用する場合は、pgvector を導入し、ユーザーとデータベースを作成
#           する
#           (拡張の有効化はアプリケーションの起動時に自動で実行される)
#           sudo apt install postgresql-17-pgvector # または brew install pgvector など
#           psql -U postgres -c "CREATE USER digitalbase WITH PASSWORD 'digitalbase';"
#           psql -U postgres -c "CREATE DATABASE digitalbase OWNER digitalbase;"
```

続いて、アプリケーションを起動します。

```
mkdir digitalbase && cd digitalbase
cat > .env <<EOF
LLM_BACKEND=vllm
DATABASE_URL=postgresql://digitalbase:digitalbase@host.docker.internal:5432/digitalbase
VLLM_BASE_URL=http://host.docker.internal:8080
VLLM_EMBED_BASE_URL=http://host.docker.internal:8081
JWT_SECRET=$(openssl rand -hex 32)
OAUTH_ENCRYPTION_KEY=$(openssl rand -hex 32)
AUTH_MODE=local
EOF
cp /path/to/license.lic ./license.lic

docker run -d --name digitalbase-app \
  -p 8000:8000 \
  --env-file .env \
  -v "$PWD":/app/data \
  --add-host=host.docker.internal:host-gateway \
  --restart unless-stopped \
  lmlight/digitalbase:latest
```

スキーマ・テーブル・初期管理者ユーザーの作成、および pgvector 拡張の有効化は、アプリケーションの起動時に自動で実行されます（PostgreSQL に pgvector が導入済みであることのみが前提です）。起動後、`http://localhost:8000` にアクセスし、`admin@local / admin123` でログインしてください。Ollama 版を使用する場合は、`.env` を `LLM_BACKEND=ollama` および `OLLAMA_BASE_URL=http://host.docker.internal:11434` に変更します。

データベースの所有者以外のユーザーで接続しており `CREATE EXTENSION` に失敗する場合のみ、スーパーユーザーで一度 `CREATE EXTENSION vector` を実行してください。

Kubernetes

専用 chart は配布していません。Docker Hub の image を引いて、自前の manifest で配備してください（env は Secret/ConfigMap、`/app/data` は PVC）。

```
lmlight/digitalbase:latest
```

2. .env 設定

`.env` ファイル位置:

- Linux / macOS: `~/local/db/.env`
- Windows: `%LOCALAPPDATA%\db\.env`
- Docker: `~/digitalbase/.env` (`install-docker.sh` が自動生成、`DB_INSTALL_DIR` で変更可)

vLLM 版（既定。GPU / Docker 向け）

環境変数	説明	デフォルト
LLM_BACKEND	バックエンド種別	vllm

環境変数	説明	デフォルト
VLLM_BASE_URL	chat server URL	<code>http://localhost:8080</code>
VLLM_EMBED_BASE_URL	embed server URL	<code>http://localhost:8081</code>
VLLM_VISION_BASE_URL	vision server URL (空欄なら chat を兼用)	(空)
VLLM_AUTO_START	起動時 vLLM server も spawn	<code>true</code> (Docker は <code>false</code>)
VLLM_CHAT_MODEL	chat model (HuggingFace ID)	<code>Qwen/Qwen3-4B</code> (4B / 32K context / 約 8GB VRAM)
VLLM_EMBED_MODEL	embed model	<code>Qwen/Qwen3-Embedding-0.6B</code>
VLLM_PYTHON	vLLM venv の python path	<code>~/.local/db/venv/bin/python</code>
VLLM_TENSOR_PARALLEL	GPU 数 (tensor parallel size)	<code>1</code>
VLLM_GPU_MEMORY_UTILIZATION_CHAT	chat GPU memory ratio (chat+embed 同 GPU 時)	<code>0.70</code>
VLLM_GPU_MEMORY_UTILIZATION_EMBED	embed GPU memory ratio	<code>0.10</code>
VLLM_GPU_MEMORY_UTILIZATION_VISION	vision GPU memory ratio (vision server 利用時)	(空)
LLM_CONTEXT_LENGTH	context window (<code>--max-model-len</code> 。両 backend 共通)	未設定 (model default = 32K)
VLLM_REASONING_PARSER	thinking/reasoning の parser (例 <code>qwen3</code>)。未設定なら vLLM default	(空)
VLLM_EXTRA_ARGS_CHAT	chat server の追加起動フラグ。 tool calling は要設定 (例 <code>--enable-auto-tool-choice --tool-call-parser hermes</code>)。 embed/vision 用は <code>VLLM_EXTRA_ARGS_EMBED</code> / <code>VLLM_EXTRA_ARGS_VISION</code>	(空)
HF_HUB_OFFLINE	<code>1</code> で network 不要 (air-gapped。model の事前 cache が必要)	(空)
DATABASE_URL	PostgreSQL 接続文字列	install script が自動生成
LICENSE_FILE_PATH	ライセンス path	<code>~/.local/db/license.lic</code> (Docker: <code>/app/data/license.lic</code>)
API_HOST / API_PORT	bind / port	<code>0.0.0.0 / 8000</code>

環境変数	説明	デフォルト
JWT_SECRET	JWT 署名（自動生成。再生成すると既存 session が無効になります）	install 時 random
OAuth_ENCRYPTION_KEY	OAuth 連携 token の暗号化鍵（変更すると既存の連携が復号できなくなるため、固定してください）	install 時 random（Bare Metal は手動設定も可）
AUTH_MODE	認証: local / ldap / oidc	local

Ollama 版（Bare Metal デスクトップ: Linux / macOS / Windows）

環境変数	説明	デフォルト
LLM_BACKEND	バックエンド種別	ollama
OLLAMA_BASE_URL	Ollama サーバ URL	http://localhost:11434
OLLAMA_AUTO_START	起動時 Ollama daemon を spawn	true（Docker は false）

その他（DB / License / JWT / OAuth / AUTH / API / LLM_CONTEXT_LENGTH）は vLLM 版と同じ。

Docker 版

install-docker.sh が .env を自動生成。docker 特有の差分:

環境変数	docker 特有設定
DATABASE_URL	同 docker network 内 PostgreSQL → digitalbase-postgres:5432
OLLAMA_BASE_URL	host の Ollama 参照 → http://host.docker.internal:11434
OLLAMA_AUTO_START	false（必須。1 container 1 process が原則で、container 内では spawn できません）
VLLM_BASE_URL	外部 vLLM 参照 → http://host.docker.internal:8080
VLLM_AUTO_START	false（必須）
LICENSE_FILE_PATH	image ENV 既定 → /app/data/license.lic（mount 経由）
FILES_DIR	image ENV 既定 → /app/data/files（mount 経由）

Cloud LLM（OpenAI / Anthropic / Gemini）を利用する場合は、すべての版で次を設定します。

```
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
GEMINI_API_KEY=AIza...
```

3. データベース

インストーラ（install-*.sh）が DB に対して行うのは bootstrap のみです（superuser でしか実行できない、次の3つ）。

- DB ユーザー作成 (**digitalbase**)
- データベース作成 (**digitalbase**、owner = 同ユーザー)
- pgvector 拡張の有効化 (**CREATE EXTENSION vector**。失敗時は RAG を無効化して続行)

接続情報は **.env** の **DATABASE_URL** で変更できます。既存の PostgreSQL / RDS 等に向ける場合はここを書き換えてください（その場合は上記 3 つを DBA に依頼します）。

schema / table / index / 初期 admin user は、backend の初回起動時に **migrations.py** が冪等に自動作成します（アップデート時の列追加・移行も自動で、再 install してもデータは保持されます）。手動操作は不要です。

schema 構成: **public**（主要 entity）/ **approval** / **helpdesk** / **vision** / **log** / **datalake** / **pgvector**

前提として、PostgreSQL が起動している必要があります（停止中は bootstrap がスキップされます）。DBA 管理や air-gapped 環境で table まで事前に投入したい場合のみ、**db_setup.sh** を superuser で実行してください（user / DB / 拡張に加えて、全 schema・table・index を一括投入します）。

4. ライセンス

配置

license.lic を取得後:

- Linux / macOS: **~/local/db/license.lic**
- Windows: **%LOCALAPPDATA%\db\license.lic**
- Docker: **~/digitalbase/license.lic** に置いて **docker restart digitalbase-app**、または admin UI から upload
- Kubernetes: Secret を **/app/data/license.lic** に mount

5. 起動・停止 / アクセス

コマンド

Edition	start	stop
Linux / macOS / Win (vLLM / Ollama)	db start	db stop
Docker (digitalbase-app)	docker start digitalbase-app	docker stop digitalbase-app

ログは、Bare Metal では **db start** を実行した前景に出力されます。Docker では **docker logs -f digitalbase-app** で確認できます。

アクセス

- ローカル: **http://localhost:8000**
- LAN: 起動時に LAN IP 表示、他 PC / モバイルからアクセス可
- 初回ログイン: **admin@local** / **admin123**（ログイン後にパスワードの変更が必要です）

6. アップデート

同じインストールコマンドを再実行します（データは保持され、**.env** は上書きされません）。

Docker の場合は `docker pull lmlight/digitalbase:latest` を実行してから `install` を再実行します (container は作り直されますが、data は volume に保持されます)。

7. アンインストール

```
# Linux / macOS (vLLM / Ollama 共通)
rm -rf ~/.local/db && sudo rm -f /usr/local/bin/db

# Docker
docker rm -f digitalbase-app digitalbase-postgres
docker network rm digitalbase-net
rm -rf ~/digitalbase
```

Windows:

```
Remove-Item -Recurse -Force "$env:LOCALAPPDATA\db"
```

8. 文字起こし (オプション)

音声・動画の文字起こしを使う場合のみ必要です。必要な環境: FFmpeg (音声・動画処理)

- Linux: `sudo apt install -y ffmpeg`
- macOS: `brew install ffmpeg`
- Windows: installer が winget で FFmpeg を導入

```
# Linux / macOS
curl -fsSL https://pub-a2cab4360f1748cab5ae1c0f12cddc0a.r2.dev/vite-
scripts/install-transcribe.sh | bash

# Windows
irm https://pub-a2cab4360f1748cab5ae1c0f12cddc0a.r2.dev/vite-scripts/install-
transcribe.ps1 | iex
```

付録

ディレクトリ構造

<code>~/.local/db/</code>	# Bare Metal の data 実体 (Docker は ~/digitalbase)
├─ <code>api</code>	# binary (= API + frontend 同梱、Bare Metal のみ)
├─ <code>.env</code>	# 設定
├─ <code>license.lic</code>	# ライセンス
├─ <code>files/</code>	# ユーザ file
├─ <code>postgres-data/</code>	# (Docker 版のみ) PG data
└─ <code>start.sh / stop.sh</code>	# (Bare Metal のみ)